

智能巡检车 PRO 版实验手册

机器人目标检测模型训练

目录

1 实验背景	2
2 实验原理	4
3 实验目的	4
4 实验环境	5
5 实验流程	5
6 实验任务	7
6.1 任务一 安装 GPU 驱动、CUDA、cuDNN	8
6.2 任务二 安装 Anaconda 及 yolov5-7.0 环境	19
6.3 任务三 准备训练数据集并进行 YOLOv5 配置	26
6.4 任务四 YOLOv5 模型训练	32
7 实验结果	37
8 实验资源释放	38
9 实验小结	38

1

实验背景

目标检测（Object Detection）的任务是找出图像中所有感兴趣的目标（物体），确定它们的类别和位置，是计算机视觉领域的核心问题之一。由于各类物体有不同的外观、形状和姿态，加上成像时光照、遮挡等因素的干扰，目标检测一直是计算机视觉领域最具有挑战性的问题。

YOLOv5 是一个非常流行的实时目标对象检测算法，它是由 Glenn Jocher 和 Ultralytics 团队开发的。YOLO 意为 "You Only Look Once"（你只看一次），这是对 YOLO 系列神经网络模型工作原理的一个简洁描述。YOLOv5 在保持了 YOLO 系列快速检测速度的同时，进一步提高了检测精度。

Yolov5 的主要特点：

1、速度快

YOLOv5 能够在多种硬件上实现快速的实时检测，包括但不限于 CPU、GPU 和 TPU。这使得它非常适合于嵌入式系统和移动设备上的应用。

2、准确度高

与之前的版本相比，YOLOv5 在多个基准数据集上都表现出了更高的准确率，特别是在小目标检测方面有显著提升。

3、易于使用

YOLOv5 提供了一个简单易用的 Python 接口，使得开发者可以很容易地训练自己的数据集，并进行模型微调。官方还提供了详细的文档和教程来帮助用户入门。

4. 模块化设计

YOLOv5 采用了更加模块化的架构,允许用户轻松地修改网络结构或添加新的组件,以适应特定的应用需求。

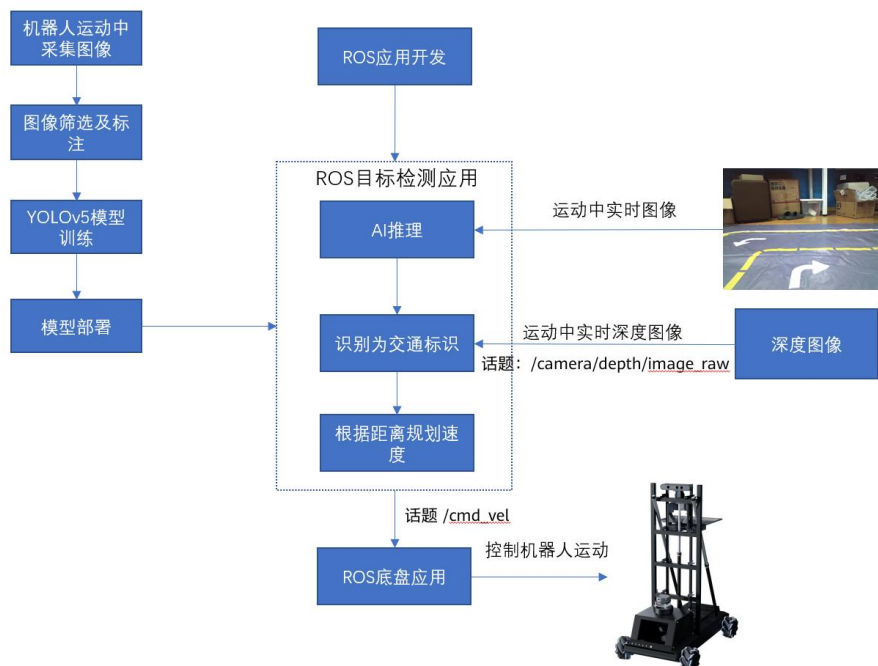
5. 支持多种框架

虽然 YOLOv5 最初是在 PyTorch 上实现的,但它也可以通过各种工具转换成其他深度学习框架的支持格式,如 TensorFlow、ONNX 等。

YOLOv5 拥有一个活跃的社区,用户可以在 GitHub 项目页面上找到大量的资源和支持。无论是新手还是经验丰富的开发者,都能在这个社区中获得帮助。

总之,YOLOv5 是一个强大的工具,适用于需要高效、精确的目标检测解决方案的各种应用场景。我们也选择 YOLOv5 作为机器人进行目标检测的神经网络算法。

在上一节中,我们介绍了机器人目标检测功能总体流程:



在上一节中,通过控制机器人在特定场景中的运动拍照,我们已经获得了所需要的图像数据,并使用 labelimg 标注工具进行了数据标注。

这一节我们介绍如何构建 YOLOv5 目标检测神经网络的训练环境,如何使用标注数据

进行模型训练，得到我们想要的 AI 模型。

对于训练 AI 模型所需要的环境设置，我们也会提供样例说明，学员可以参考设置自己的训练环境。但由于每个人所拥有的硬件环境可能不同，样例供参考，学员可以从互联网上获得足够的信息完成环境准备工作。

2

实验原理

1. 模型训练需要一台安装有 Nvidia GPU 的 X86 主机，GPU 可以是 GeForce GTX 1070 型号的，也可以是 GeForce RTX 3060 型号的，甚至是 GeForce RTX 4090 型号的等等，GPU 的算力越高，模型训练的速度越快。
2. X86 主机上可以安装 Ubuntu18.04 操作系统，并在此基础上安装 GPU 的驱动、Cuda 及 cuDNN 等，准备好基础环境。
3. 在基础环境上，安装 Anaconda，并创建 YOLOv5 虚拟环境。
4. 下载 YOLOv5 版本，安装所需依赖。
5. 拆分标注数据集，并配置 YOLOv5 训练参数。
6. 进行 YOLOv5 模型训练，并对得到的模型效果进行评估。

3

实验目的

1. 了解如何准备 YOLOv5 模型训练软硬件环境。
2. 掌握 YOLOv5 的下载及安装过程。
3. 掌握对数据集的拆分方法。
4. 掌握对 YOLOv5 的训练配置及训练过程。
5. 掌握对 YOLOv5 模型效果的测试方法。

4

实验环境

1. 硬件：x86 64 位台式主机+Nvidia GPU (1070 及以上，比如 3060 或者 4090 等)。
2. 代码开发工具版本：PyCharm 2023.1.1 (Community Edition)。
3. 操作系统版本：Ubuntu18.04。
4. Git 版本：2.17.1 及以上。
5. 开发语言：Python 3.8 及以上 ， C++ 11 及以上。

5

实验流程

实验整体流程图如下所示：



首先，我们假定用户已经准备好了配置了 Nvidia GPU 的 x86 台式机。

在前面 2.2 章节我们介绍过如何在虚拟机上安装 Ubuntu20.04，这里对于如何在 x86 台式机上如何安装 Ubuntu18.04 操作系统，就不再做介绍，学员可以参考网络上的教程自行完成系统安装。

在此基础上，还需要进行如下工作：

- (1) 在 Ubuntu18.04 中安装 GPU 驱动、Cuda 及 cuDNN，准备好基础环境。
- (2) 在基础环境中安装 Anaconda、YOLOv5 虚拟环境，下载 YOLOv5 及安装相关依赖。
- (3) 拆分完整的标注数据集为训练集、验证集和测试集。
- (4) 配置 YOLOv5 训练参数，训练 YOLOv5 模型。

(5) 测试 YOLOv5 模型效果。

6

实验任务

实验整体流程图如下所示：



首先，我们假定用户已经准备好了配置了 Nvidia GPU 的 x86 台式机。

在前面 2.2 章节我们介绍过如何在虚拟机上安装 Ubuntu20.04，这里对于如何在 x86 台式机上如何安装 Ubuntu18.04 操作系统，就不再做介绍，学员可以参考网络上的教程自行完成系统安装。

在此基础上，还需要进行如下工作：

- (6) 在 Ubuntu18.04 中安装 GPU 驱动、Cuda 及 cuDNN，准备好基础环境。
- (7) 在基础环境中安装 Anaconda、YOLOv5 虚拟环境，下载 YOLOv5 及安装相关依赖。
- (8) 拆分完整的标注数据集为训练集、验证集和测试集。
- (9) 配置 YOLOv5 训练参数，训练 YOLOv5 模型。
- (10) 测试 YOLOv5 模型效果。

6.1 任务一 安装 GPU 驱动、CUDA、cuDNN

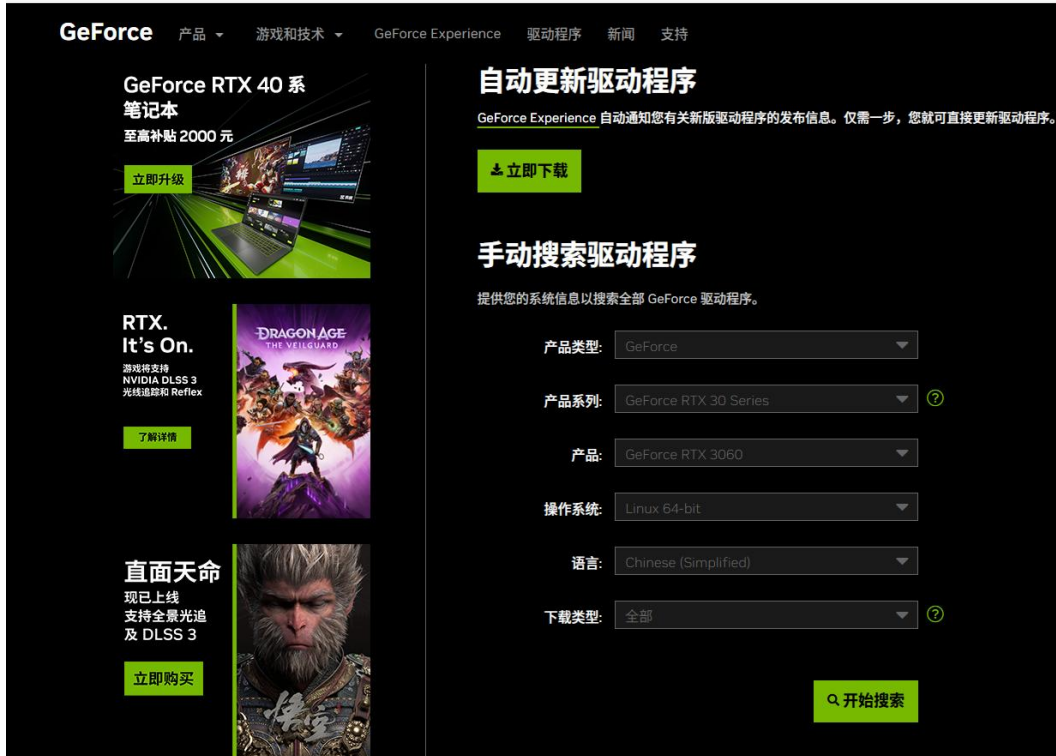
1. 查看本机显卡能够配置的驱动信息

比如，某台测试机器的执行结果如下，说明推荐 470 版本的驱动。

```
$ ubuntu-drivers devices
== /sys/devices/pci0000:00/0000:00:01.0/0000:01:00.0 ==
modalias : pci:v000010DEd00002504sv00001458sd00004072bc03sc00i00
vendor    : NVIDIA Corporation
manual_install: True
driver     : nvidia-driver-470 - distro non-free recommended
driver     : nvidia-driver-470-server - distro non-free
driver     : nvidia-driver-515 - distro non-free
driver     : nvidia-driver-530 - distro non-free
driver     : nvidia-driver-515-server - distro non-free
driver     : nvidia-driver-510 - distro non-free
driver     : nvidia-driver-525-server - distro non-free
driver     : nvidia-driver-525 - distro non-free
driver     : xserver-xorg-video-nouveau - distro free builtin
```

2. 访问 Nvidia 显卡驱动官网，选择符合你的显卡的驱动进行下载

<https://www.nvidia.cn/geforce/drivers/>



比如，3060 显卡，选择了版本为 470 的驱动，并下载：

NVIDIA-Linux-x86_64-470.129.06.run

也可以从本系列课程提供的资源目录下的“tools”路径下获取。

3. 使用如下指令删除系统中已存在的驱动

```
sudo apt-get remove --purge nvidia*
```

4. 将 nouveau 驱动禁用

使用命令打开配置文件/etc/modprobe.d/blacklist-nouveau.conf

```
sudo gedit /etc/modprobe.d/blacklist.conf
```

添加以下内容后保存：

```
blacklist nouveau
blacklist lbm-nouveau
options nouveau modeset=0
alias nouveau off
alias lbm-nouveau off
```

5. 检查 nouveau 配置文件

```
echo options nouveau modeset=0 | sudo tee -a  
/etc/modprobe.d/nouveau-kms.conf
```

6. 更新配置文件

```
sudo update-initramfs -u
```

7. 重启系统

```
reboot
```

8. 验证 nouveau 是否已禁用，如果没有返回说明已经被禁用

```
lsmod | grep nouveau
```

9. 查看操作系统的发行版本号

```
$ uname -r  
5.4.0-150-generic
```

10. 获取 kernel source 命令

命令中的 x.x.x-x 是上一步中获得到的，比如:5.4.0-150

```
sudo apt-get install linux-source  
  
sudo apt-get install linux-headers-x.x.x-x-generic
```

11. 给驱动文件赋予执行权限

```
sudo chmod a+x NVIDIA-Linux-x86_64-470.129.06.run
```

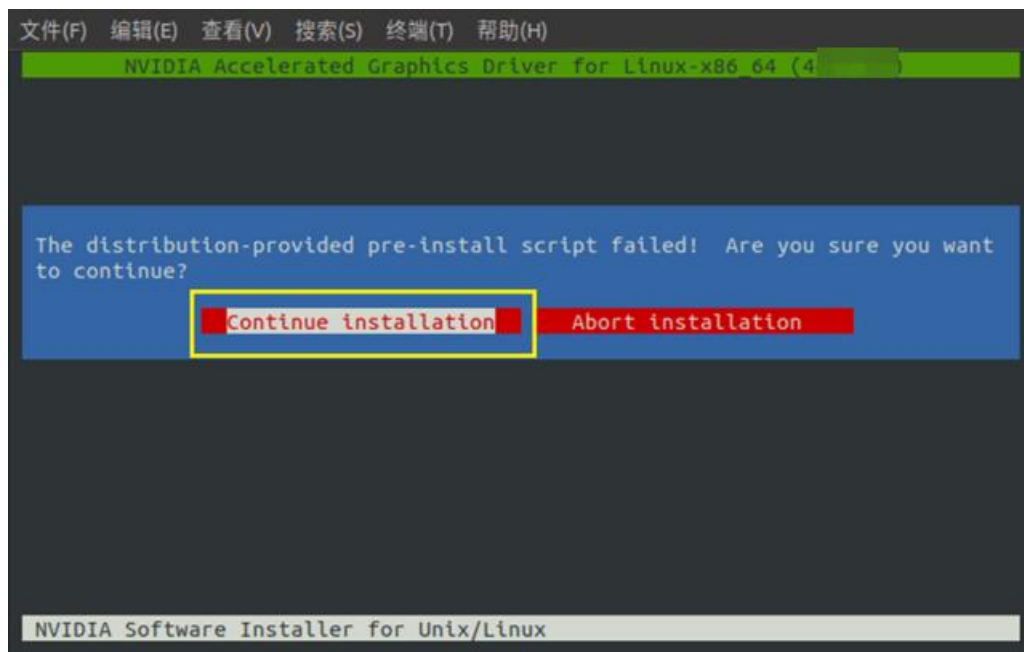
12. 安装驱动

```
sudo ./NVIDIA-Linux-x86_64-470.129.06.run -no-x-check -no-nouveau-check  
-no-opengl-files
```

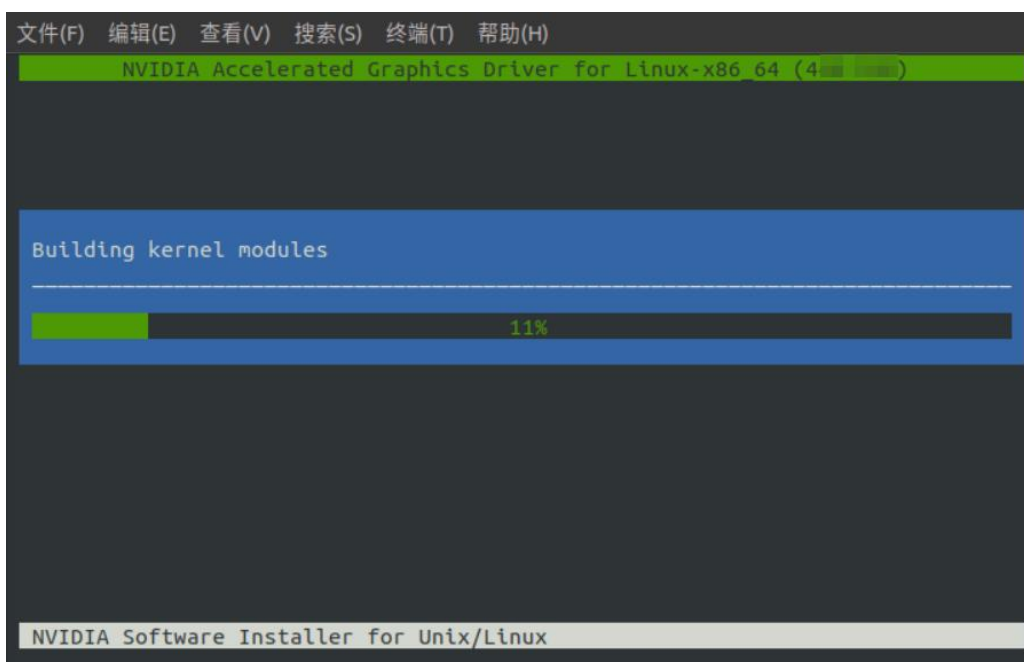
参数-no-x-check 表示安装驱动时关闭 x 服务。

参数-no-nnouveau-check 表示安装驱动时禁用 nouveau。

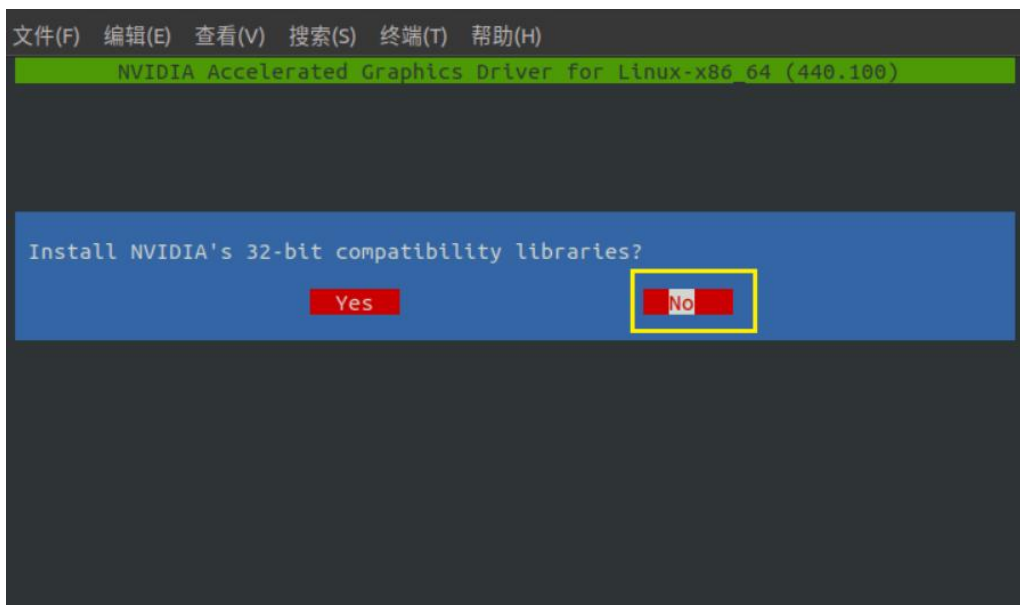
参数-no-opengl-files 表示只安装驱动文件，不安装 OpenGL 文件。



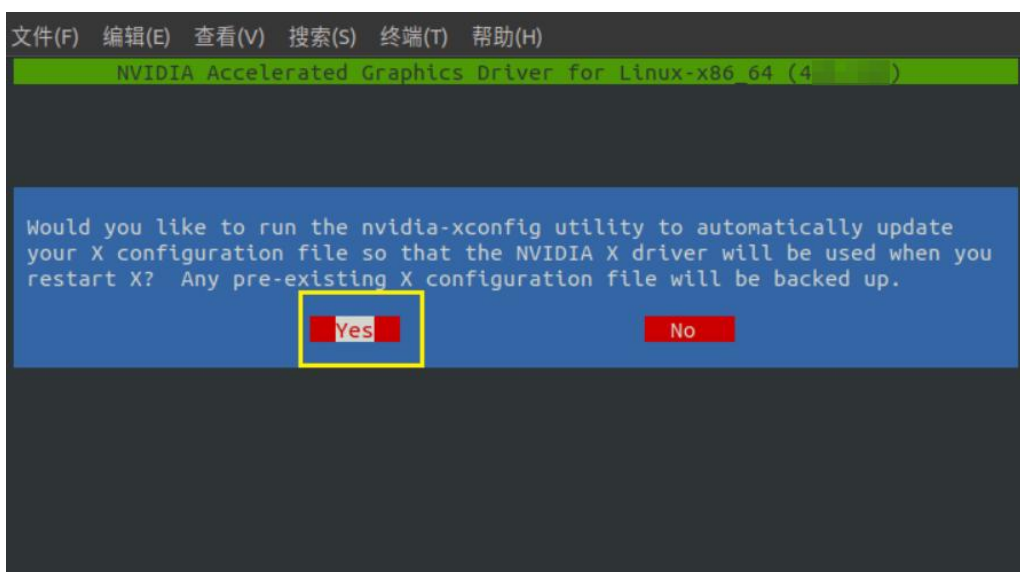
出现如上图界面，选择“Continue installation”。



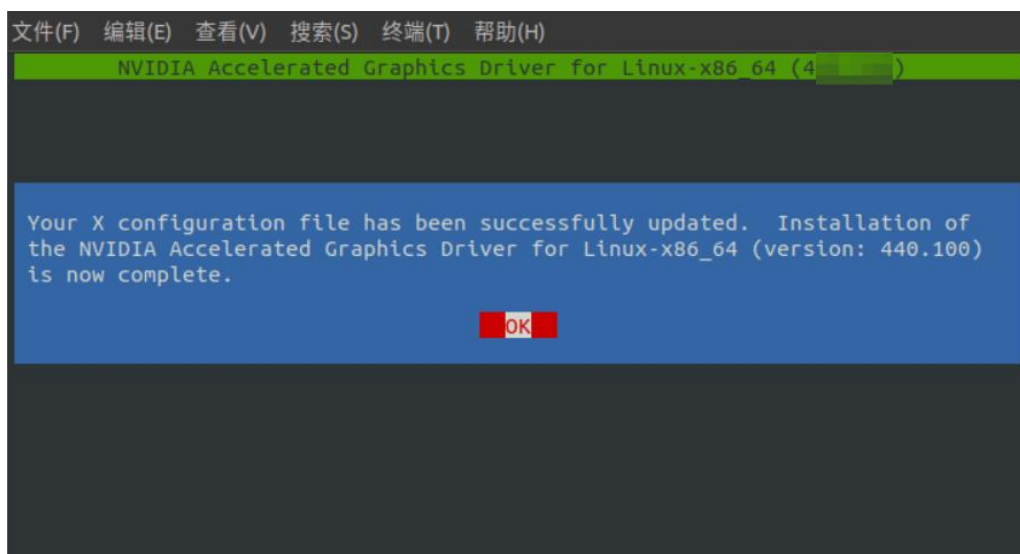
构建内核模块中。



出现上述画面，选择 “No” 。



出现上述画面，选择 “Yes” 。



出现上述画面，选择“OK”，完成驱动安装。

13. 检验驱动是否安装成功

```
lebox100@lebox100:~$ nvidia-smi
Sat Nov 9 20:27:41 2024
+-----+
| NVIDIA-SMI 470.129.06    Driver Version: 470.129.06    CUDA Version: 11.4    |
+-----+-----+
| GPU Name               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                               |                  |              MIG M. |
+-----+-----+
| 0  NVIDIA GeForce ...  Off      | 00000000:01:00.0 On  |          N/A         |
| 0%   44C    P8      25W / 170W | 249MiB / 12050MiB |      18%    Default  |
|                               |                  |              N/A     |
+-----+-----+

Processes:
+-----+-----+
| GPU  GI  CI           PID  Type  Process name                        GPU Memory |
|   ID  ID  ID             |                  | Usage       |
+-----+-----+
| 0    N/A N/A        1754    G   /usr/lib/xorg/Xorg                  18MiB      |
| 0    N/A N/A        1936    G   /usr/bin/gnome-shell                69MiB      |
| 0    N/A N/A        9097    G   /usr/lib/xorg/Xorg                  125MiB     |
| 0    N/A N/A        9222    G   /usr/bin/gnome-shell                32MiB      |
+-----+-----+
```

14. 重启系统

```
reboot
```

15. 从 nvidia-smi 命令可以看到显卡驱动适配的 CUDA 版本


```
lebox100@lebox100:~$ nvidia-smi
Sat Nov 9 20:27:41 2024

+-----+
| NVIDIA-SMI 470.129.06 | Driver Version: 470.129.06 | CUDA Version: 11.4 |
+-----+
| GPU  Name           Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+-----+=====+
| 0   NVIDIA GeForce ...   Off   | 00000000:01:00:0 | On          N/A       |
| 0%   44C    P8     25W / 170W | 249MiB / 12050MiB | 18%        Default  |
|=====+-----+=====+
|                          MIG M. |
|                          N/A       |
+-----+

Processes:
+-----+
| GPU  GI  CI       PID   Type   Process name                      GPU Memory |
|   ID  ID  ID             |                 | Usage     |
+-----+
| 0   N/A N/A       1754  G    /usr/lib/xorg/Xorg                 18MiB     |
| 0   N/A N/A       1936  G    /usr/bin/gnome-shell               69MiB     |
| 0   N/A N/A       9097  G    /usr/lib/xorg/Xorg                 125MiB    |
| 0   N/A N/A       9222  G    /usr/bin/gnome-shell               32MiB     |
+-----+
```

测试用的这台机器对应的 CUDA 版本是 11.4。

16. 访问 CUDA 官网，获取 CUDA 版本

<https://developer.nvidia.com/cuda-toolkit-archive>

CUDA Toolkit 11.4 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux	Windows
Architecture	x86_64	ppc64le arm64-sbsa
Distribution	CentOS	Debian Fedora OpenSUSE RHEL SLES Ubuntu WSL-Ubuntu
Version	18.04	20.04
Installer Type	deb (local)	deb (network) runfile (local)

Download Installer for Linux Ubuntu 18.04 x86_64

The base installer is available for download below.

> Base Installer

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cuda/11.4.0/local_installers/cuda_11.4.0_470.42.01_linux.run
$ sudo sh cuda_11.4.0_470.42.01_linux.run
```

The CUDA Toolkit contains Open-Source Software. The source code can be found [here](#).
The checksums for the installer and patches can be found in [Installer Checksums](#).
For further information, see the [Installation Guide for Linux](#) and the [CUDA Quick Start Guide](#).

通过 wget 指令获取 CUDA 安装包：cuda_11.4.0_470.42.01_linux.run。

也可以从本系列课程提供的资源目录下的“tools”路径下获取。

17. 执行指令安装 CUDA

```
sudo sh cuda_11.4.0_470.42.01_linux.run
```

出现如下文档，输入 accept

```
hingwenwong@hingwenwong-MS-7C73: /media/hingwenwong/HDD/HinGwenWo
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

different security, privacy, accessibility, availability, and
reliability standards relative to commercial versions of
NVIDIA software and materials. Use of a pre-release SDK may
result in unexpected results, loss of data, project delays or
other unpredictable damage or loss.

You may use a pre-release SDK at your own risk, understanding
that pre-release SDKs are not intended for use in production
or business-critical systems.

NVIDIA may choose not to make available a commercial version
of any pre-release SDK. NVIDIA may also choose to abandon
development and terminate the availability of a pre-release
SDK at any time without liability.

1.1.5. Updates

NVIDIA may, at its option, make available patches, workarounds
or other updates to this SDK. Unless the updates are provided

Do you accept the above EULA? (accept/decline/quit):
accept
```

回车续执行。

```
CUDA-Installer
- [ ] Driver          取消选中!!!
  [ ] 440.33.01
+ [X] CUDA Toolkit 10.2
  [X] CUDA Samples 10.2
  [X] CUDA Demo Suite 10.2
  [X] CUDA Documentation 10.2
Options
Install

Up/Down: Move | Left/Right: Expand | 'Enter': Select | 'A': Advanced options
```

取消选中 Driver，因为前面已经安装了显卡驱动，此处不取消掉后面会导致冲突报错。


```
CUDA Installer
- [ ] Driver
  [ ] 440.33.01
+ [X] CUDA Toolkit 10.2
  [X] CUDA Samples 10.2
  [X] CUDA Demo Suite 10.2
  [X] CUDA Documentation 10.2
Options
Install

Up/Down: Move | Left/Right: Expand | 'Enter': Select | 'A': Advanced options
```

选择 “Install” 进行安装。

等待一段时间后，会弹出如下告警信息，这是因为取消了安装 Driver 导致的，直接忽略即可。

```
=====
= Summary =
=====

Driver:   Not Selected
Toolkit:  Installed in /usr/local/cuda-10.2/
Samples:  Installed in /home/hingwenwong/, but missing recommended libraries

Please make sure that
- PATH includes /usr/local/cuda-10.2/bin
- LD_LIBRARY_PATH includes /usr/local/cuda-10.2/lib64, or, add /usr/local/cuda-10.2/lib64 to /etc/ld.so.conf and run ldconfig as root

To uninstall the CUDA Toolkit, run cuda-uninstaller in /usr/local/cuda-10.2/bin

Please see CUDA_Installation_Guide_Linux.pdf in /usr/local/cuda-10.2/doc/pdf for detailed information on setting up CUDA.
***WARNING: Incomplete installation! This installation did not install the CUDA Driver. A driver of version at least 440.
00 is required for CUDA 10.2 functionality to work.
To install the driver using this installer, run the following command, replacing <CudaInstaller> with the name of this ru
n file:
    sudo <CudaInstaller>.run --silent --driver
Logfile is /var/log/cuda-installer.log
```

18. 修改环境变量

在 ~/.bashrc 文件中写入如下内容，并保存退出：

```
export PATH=$PATH:/usr/local/cuda-11.4/bin
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda-11.4/lib64
export CUDA_HOME=$CUDA_HOME:/usr/local/cuda
```

执行如下命令，生效配置：

```
source ~/.bashrc
```

19. 使用 nvcc 命令测试 CUDA 安装成功

测试 CUDA 命令，显示如下内容说明安装成功。

```
$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Wed_Jun__2_19:15:15_PDT_2021
Cuda compilation tools, release 11.4, V11.4.48
Build cuda_11.4.r11.4/compiler.30033411_0
```

21. 访问 cuDNN 官网，下载匹配 CUDA 版本的 cuDNN.

<https://developer.nvidia.com/rdp/cudnn-archive>



也可以从本系列课程提供的资源目录下的“tools”路径下获取。

22. 下载完成后进行解压缩，并进入 cuda 目录：

```
lebox100@lebox100:~/nvidia/cuda$ tree . -L 2
.
├── include
│   ├── cudnn_adv_infer.h
│   ├── cudnn_adv_infer_v8.h
│   ├── cudnn_adv_train.h
│   ├── cudnn_adv_train_v8.h
│   ├── cudnn_backend.h
│   ├── cudnn_backend_v8.h
│   ├── cudnn_cnn_infer.h
│   ├── cudnn_cnn_infer_v8.h
│   ├── cudnn_cnn_train.h
│   ├── cudnn_cnn_train_v8.h
│   ├── cudnn.h
│   ├── cudnn_ops_infer.h
│   ├── cudnn_ops_infer_v8.h
│   ├── cudnn_ops_train.h
│   ├── cudnn_ops_train_v8.h
│   ├── cudnn_v8.h
│   ├── cudnn_version.h
│   └── cudnn_version_v8.h
├── lib64
│   ├── libcudnn_adv_infer.so -> libcudnn_adv_infer.so.8
│   ├── libcudnn_adv_infer.so.8 -> libcudnn_adv_infer.so.8.2.4
│   ├── libcudnn_adv_infer.so.8.2.4
│   ├── libcudnn_adv_train.so -> libcudnn_adv_train.so.8
│   ├── libcudnn_adv_train.so.8 -> libcudnn_adv_train.so.8.2.4
│   ├── libcudnn_adv_train.so.8.2.4
│   ├── libcudnn_cnn_infer.so -> libcudnn_cnn_infer.so.8
│   ├── libcudnn_cnn_infer.so.8 -> libcudnn_cnn_infer.so.8.2.4
│   ├── libcudnn_cnn_infer.so.8.2.4
│   ├── libcudnn_cnn_infer_static.a
│   ├── libcudnn_cnn_infer_static_v8.a -> libcudnn_cnn_infer_static.a
│   ├── libcudnn_cnn_train.so -> libcudnn_cnn_train.so.8
│   ├── libcudnn_cnn_train.so.8 -> libcudnn_cnn_train.so.8.2.4
│   ├── libcudnn_cnn_train.so.8.2.4
│   ├── libcudnn_cnn_train_static.a
│   ├── libcudnn_cnn_train_static_v8.a -> libcudnn_cnn_train_static.a
│   ├── libcudnn_ops_infer.so -> libcudnn_ops_infer.so.8
│   ├── libcudnn_ops_infer.so.8 -> libcudnn_ops_infer.so.8.2.4
│   ├── libcudnn_ops_infer.so.8.2.4
│   ├── libcudnn_ops_train.so -> libcudnn_ops_train.so.8
│   ├── libcudnn_ops_train.so.8 -> libcudnn_ops_train.so.8.2.4
│   ├── libcudnn_ops_train.so.8.2.4
│   ├── libcudnn.so -> libcudnn.so.8
│   ├── libcudnn.so.8 -> libcudnn.so.8.2.4
│   ├── libcudnn.so.8.2.4
│   ├── libcudnn_static.a
│   └── libcudnn_static_v8.a -> libcudnn_static.a
└── NVIDIA_SLA_cuDNN_Support.txt

2 directories, 46 files
```

23. 使用如下指令，对文件进行复制

```
sudo cp include/cudnn.h /usr/local/cuda/include/  
sudo cp lib64/libcudnn* /usr/local/cuda/lib64/  
sudo chmod a+r /usr/local/cuda/include/cudnn.h  
sudo chmod a+r /usr/local/cuda/lib64/libcudnn*
```

24. 验证 cudnn:

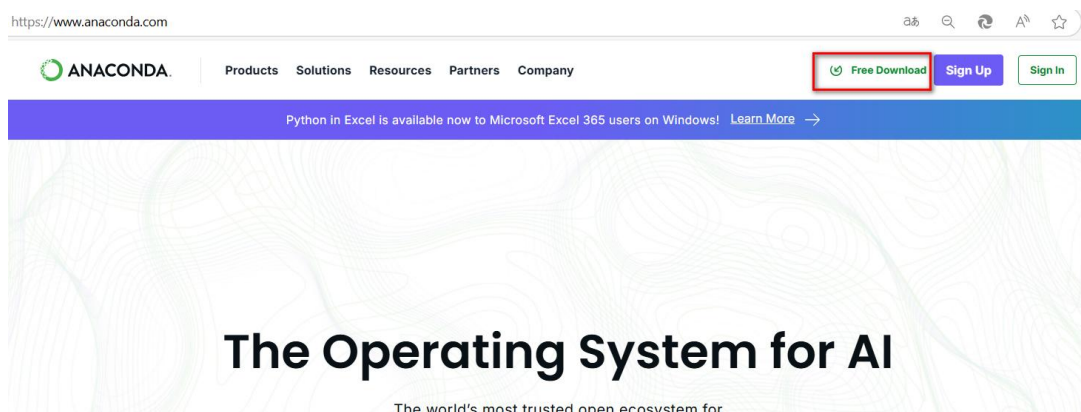
```
$ cat /usr/local/cuda/include/cudnn_version.h |grep CUDNN_MAJOR -A 2  
#define CUDNN_MAJOR 8  
#define CUDNN_MINOR 2  
#define CUDNN_PATCHLEVEL 4  
--  
#define CUDNN_VERSION (CUDNN_MAJOR * 1000 + CUDNN_MINOR * 100 +  
CUDNN_PATCHLEVEL)  
  
#endif /* CUDNN_VERSION_H */
```

看到 cudnn 的版本信息符合预期，即证明安装成功。

6.2 任务二 安装 Anaconda 及 yolov5-7.0 环境

1. 去 Anaconda 官网下载安装包

官网地址: <https://www.anaconda.com/>



点击页面上的“Free Download”链接

Distribution

Free Download*

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- ✓ Easily search and install thousands of data science, machine learning, and AI packages
- ✓ Manage packages and environments from a desktop application or work from the command line
- ✓ Deploy across hardware and software platforms
- ✓ Distribution installation on Windows, MacOS, or Linux

*Use of Anaconda's Offerings at an organization of more than 200 employees requires a Business or Enterprise license. [See Pricing](#)

Provide email to download Distribution

Email Address:

- ☐ Agree to receive communication from Anaconda regarding relevant content, products, and services. I understand that I can revoke this consent [here](#) at any time.

By continuing, I agree to Anaconda's [Privacy Policy](#) and [Terms of Service](#).

Submit >

Skip registration

点击“Skip registration”按钮，进入下载页面：

Download Anaconda Distribution or [Miniconda](#) by choosing the proper installer for your machine. Learn the difference from our [Documentation](#).

Anaconda Installers

Download



Windows

Python 3.12

64-Bit Graphical Installer (912.3M)



Mac

Python 3.12

64-Bit (Apple silicon) Graphical Installer (704.7M)

64-Bit (Apple silicon) Command Line Installer (707.3M)

64-Bit (Intel chip) Graphical Installer (734.7M)

64-Bit (Intel chip) Command Line Installer (731.2M)



Linux

Python 3.12

64-Bit (x86) Installer (1007.9M)

64-Bit (AWS Graviton2 / ARM64) Installer (800.6M)

64-bit (Linux on IBM Z & LinuxONE) Installer (425.8M)

选择版本进行下载。

如果你想下载旧版本的 Anaconda，可以到清华大学开源软件镜像站进行下载；

<https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/>

2. 下载好版本，就可以进行安装：

```
bash Anaconda3 版本号.sh
```

```
root@localhost:~# bash Anaconda3-4.3.0-Linux-x86_64.sh

Welcome to Anaconda3

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>> 
```

然后就是一步步按 Enter 即可。最后一步需要输入 yes

```
lib sodium
  A software library for encryption, decryption, signatures, password
  and more.

pynacl
  A Python binding to the Networking and Cryptography library, a crypt
  y with the stated goal of improving usability, security and speed.

Last updated May 20, 2020

Do you accept the license terms? [yes|no]
[no] >>>
Please answer 'yes' or 'no':
>>>
Please answer 'yes' or 'no':
>>>
Please answer 'yes' or 'no':
>>>
Please answer 'yes' or 'no':
>>>
Please answer 'yes' or 'no':
>>> yes
```

可以选择安装路径，直接输入，也可以保持默认路径，直接按 Enter

```
Anaconda3 will now be installed into this location:
/home/test/anaconda3

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/home/test/anaconda3] >>> 
```

如下界面询问是否加入环境变量，输入 yes

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
wurlitzer      pkgs/main/linux-64::wurlitzer-2.0.1-py38_0
xlrd           pkgs/main/noarch::xlrd-1.2.0-py_0
xlswriter      pkgs/main/noarch::xlswriter-1.2.9-py_0
xlwt           pkgs/main/linux-64::xlwt-1.3.0-py38_0
xmldict        pkgs/main/noarch::xmldict-0.12.0-py_0
xz             pkgs/main/linux-64::xz-5.2.5-h7b6447c_0
yaml           pkgs/main/linux-64::yaml-0.2.5-h7b6447c_0
yapf           pkgs/main/noarch::yapf-0.30.0-py_0
zeromq         pkgs/main/linux-64::zeromq-4.3.2-he6710b0_2
zict           pkgs/main/noarch::zict-2.0.0-py_0
zipp           pkgs/main/noarch::zipp-3.1.0-py_0
zlib           pkgs/main/linux-64::zlib-1.2.11-h7b6447c_3
zope           pkgs/main/linux-64::zope-1.0-py38_1
zope.event     pkgs/main/linux-64::zope.event-4.4-py38_0
zope.interface pkgs/main/linux-64::zope.interface-4.7.1-py38h7b6447c_0
zstd           pkgs/main/linux-64::zstd-1.4.5-h0b5b093_0

Preparing transaction: done
Executing transaction: done
installation finished.
Do you wish the installer to initialize Anaconda3
by running conda init? [yes/no]
[no] >>> yes
```

看到如下界面，说明安装完成

```
==> For changes to take effect, close and re-open your current shell. <==

If you'd prefer that conda's base environment not be activated on startup,
set the auto_activate_base parameter to false:

conda config --set auto_activate_base false

Thank you for installing Anaconda3!

=====

Working with Python and Jupyter notebooks is a breeze with PyCharm
Professional! Code completion, Notebook debugger, VCS support, SSH, Docker,
Databases, and more!
```

关闭当前终端重开一个，可以看到进入了 Anaconda 的 base 环境，输入 conda --version 测试，打印出版版本号即表明安装成功！

```
(base) lebox100@lebox100:~$ conda --version
conda 23.3.1
```

3. 在 conda 中创建虚拟环境

```
conda create -n yolov5-7 python=3.8
```

查看虚拟环境

```
(base) lebox100@lebox100:~ $ conda env list
# conda environments:
#
```

```
yolov5-7 /home/lebox100/.conda/envs/yolov5-7
base * /home/lebox100/miniconda3
```

激活刚刚创建的 yolov5-7 的虚拟环境

```
(base) lebox100@lebox100:~$ conda activate yolov5-7
(yolov5-7) lebox100@lebox100:~$
```

可以看到 shell 提示符前面出现了(yolov5-7)，说明激活了该环境。

4. 下载 YOLOv5 源码

从 Github 上下载 yolov5-7.0 的源码：

<https://github.com/ultralytics/yolov5/tree/v7.0>

也可以从本系列课程提供的资源目录下的“tools”路径下获取。

5. 解压缩源码包

```
unzip yolov5-7.0.zip
```

6. 安装 yolov5-7 的依赖包

在 yolov5-7.0 目录中，安装 yolov5-7.0 所依赖的包：

```
(yolov5-7) lebox100@lebox100:~/yolov5-7.0$ pip3 install -r requirements.txt -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

可以看到，pip3 按 requirements.txt 文件中的依赖项依次安装依赖包：

```
(yolov5-7) lebox100@lebox100:~/yolov5-7.0$ pip3 install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting gitpython (from -r requirements.txt (line 5))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/99/bd/cc3a402a6439c15c3d429433e13042b915bbaeb54edc457c723931fed3f/GitPython-3.1.43-py3-none-any.whl (207 kB)
Collecting ipython (from -r requirements.txt (line 6))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/8d/07/0fe103906cd81bc42d3b0175b55349f07dcaef47d6451131cf8dd070bb2/lpython-8.12.3-py3-none-any.whl (798 kB)
Collecting matplotlib>=3.2.2 (from -r requirements.txt (line 7))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/30/33/cc27211d2ffee4fd7402dca137b6e8a03f6dcae3d4be8d0ad5068555561/matplotlib-3.7.5-cp38-manylinux_2_12_x86_64.whl (9.2 MB)
Collecting numpy>=1.18.5 (from -r requirements.txt (line 8))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/98/5d/5738903efeecb73e51eb44feafba32b2081263d40c5043568ff60faf/numpy-1.24.4-cp38-cp38-manylinux_2_17_x86_64.whl (17.3 MB)
Collecting opencv-python>=4.1.1 (from -r requirements.txt (line 9))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/2f/44/d2537f47d7f7cfba96b8d80e3ec18e7ee1681056d4b0a9c8d983983e4d5/opencv_python-4.10.0.84-cp37-ab13-manylinux_2_17_x86_64.whl (62.5 MB)
Collecting pillow>=7.1.2 (from -r requirements.txt (line 10))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/84/4c/69bbd9e436ac22f9ed193a2b64f64d68fcb9f4106249dc7ed4889907b/pillow-10.4.0-cp38-cp38-manylinux_2_17_x86_64.whl (4.4 MB)
Collecting psutil (from -r requirements.txt (line 11))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/58/4d/0245e6f76a93c98aabb285a43ea71ff1b171bcd90c9d23bfbf81f7021fb233/psutil-6.1.0-cp36-ab13-manylinux_2_12_x86_64.whl (287 kB)
Collecting pyyaml>=5.3.1 (from -r requirements.txt (line 12))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/fd/7f/2c3697bba5d4a5cc2afe81826d73dfae5f049458e44732c7a0938baa673/PyYAML-6.0.2-cp38-cp38-manylinux_2_17_x86_64.whl (746 kB)
Collecting requests>=2.23.0 (from -r requirements.txt (line 13))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/f9/9b/335f9764261e915ed497fcdeb11df50fd07b7f257d4a6a2a86d80da4d54/requests-2.32.3-py3-none-any.whl (64 kB)
Collecting scipy>=1.4.1 (from -r requirements.txt (line 14))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/09/f0/fb7a9548e48b687bb0f2fa81d71ab9cfc548d365046ca1c791e240b99d/scipy-1.10.1-cp38-cp38-manylinux_2_17_x86_64.whl (34.5 MB)
Collecting thop>=0.1.1 (from -r requirements.txt (line 15))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/bb/0f/772beeab4ff5221dc47127c80f8834b4dc0cb30f6ba91c0b1d04a1233403/thop-0.1.1.post2209072238-py3-none-any.whl (15 kB)
Collecting torch>=1.7.0 (from -r requirements.txt (line 16))
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/a9/71/45aac46b75742e8d2d6f9fc2b612223b5e361158b2ed673b2c21b5387/torch-2.4.1-cp38-cp38-manylinux1_x86_64.whl (797.1 MB)
```


这个过程需要一段时间，耐心等待。

7. 下载预训练模型 yolov5s.pt

从如下路径下载预训练模型 yolov5s.pt 模型文件到 yolov5-7.0 目录：

<https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5s.pt>

也可以从本系列课程提供的资源目录下的“tools”路径下获取。

8. 执行预训练模型推理：

```
(yolov5-7) lebox100@lebox100:~/yolov5-7.0$ python3 detect.py

detect:  weights=yolov5s.pt,  source=data/images,  data=data/coco128.yaml,
imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=,
view_img=False,  save_txt=False,  save_conf=False,  save_crop=False,
nosave=False,  classes=None,  agnostic_nms=False,  augment=False,
visualize=False, update=False, project=runs/detect, name=exp, exist_ok=False,
line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False,
vid_stride=1

YOLOv5 2022-11-22 Python-3.8.20 torch-2.4.1+cu121 CPU

/home/lebox100/yolov5-7.0/models/experimental.py:79: FutureWarning: You are
using `torch.load` with `weights_only=False` (the current default value), which uses
the default pickle module implicitly. It is possible to construct malicious pickle data
which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be flipped
to `True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend
```

you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
ckpt = torch.load(attempt_download(w), map_location='cpu') # load
```

Fusing layers...

YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs

image 1/2 /home/lebox100/yolov5-7.0/data/images/bus.jpg: 640x480 4 persons, 1 bus, 45.2ms

image 2/2 /home/lebox100/yolov5-7.0/data/images/zidane.jpg: 384x640 2 persons, 2 ties, 37.4ms

Speed: 0.3ms pre-process, 41.3ms inference, 0.4ms NMS per image at shape (1, 3, 640, 640)

Results saved to runs/detect/exp2

可以看到推理成功，结果保存到了 runs/detect/exp2 目录。

进入 runs/detect/exp2 目录，使用如下命令查看推理图片：

```
xdg-open runs/detect/exp2/zidane.jpg
```



可以看到预训练模型可以对图片中的人和领结进行目标检测。

至此，yolov5-7.0 的环境就准备好了，接下来就可以准备数据集进行模型训练了。

6.3 任务三 准备训练数据集并进行 YOLOv5 配置

在上一个章节中，我们介绍了通过机器人的运动获取地图标识的数据，并对这些地图中的交通标识使用 lableimg 工具进行了标注。



在这个数据集中，一共三种类别的图像：turnright、turnleft、turnaround，分别对应了左转标识、右转标识、回头标识。在实际的应用中，机器人会根据检测到的标识来控制自己的行动方向。

接下来我们就把这些数据进行数据集的拆分，用于后面的 YOLOv5 模型训练。

1. 我们把标注数据放到 TRAFFIC 目录中：

```
(yolov5-7) lebox100@lebox100:~/data$ ls TRAFFIC/  
classes.txt  images  labels  labels.cache
```



```
(yolov5-7) lebox100@lebox100:~/data$ ls TRAFFIC/images/
20240617185734.jpg 20240617185957.jpg 20240617190209.jpg 20240617190210.jpg
20240617185736.jpg 20240617190000.jpg 20240617190211.jpg 20240617190212.jpg
20240617185738.jpg 20240617190003.jpg 20240617190216.jpg 20240617190217.jpg
20240617185741.jpg 20240617190005.jpg 20240617190218.jpg 20240617190219.jpg
20240617185744.jpg 20240617190007.jpg 20240617190222.jpg 20240617190223.jpg
20240617185746.jpg 20240617190009.jpg 20240617190227.jpg 20240617190228.jpg
20240617185748.jpg 20240617190011.jpg 20240617190230.jpg 20240617190231.jpg
20240617185749.jpg 20240617190014.jpg 20240617190231.jpg 20240617190232.jpg
20240617185753.jpg 20240617190018.jpg 20240617190233.jpg 20240617190234.jpg
20240617185759.jpg 20240617190021.jpg 20240617190238.jpg 20240617190239.jpg
20240617185804.jpg 20240617190024.jpg 20240617190240.jpg 20240617190241.jpg
20240617185807.jpg 20240617190028.jpg 20240617190242.jpg 20240617190243.jpg
20240617185810.jpg 20240617190033.jpg 20240617190244.jpg 20240617190245.jpg
20240617185814.jpg 20240617190036.jpg 20240617190245.jpg 20240617190246.jpg
20240617185817.jpg 20240617190042.jpg 20240617190248.jpg 20240617190249.jpg
20240617185819.jpg 20240617190044.jpg 20240617190252.jpg 20240617190253.jpg
20240617185821.jpg 20240617190050.jpg 20240617190253.jpg 20240617190254.jpg
20240617185824.jpg 20240617190055.jpg 20240617190256.jpg 20240617190257.jpg
20240617185827.jpg 20240617190126.jpg 20240617190258.jpg 20240617190259.jpg
```

```
(yolov5-7) lebox100@lebox100:~/data$ ls TRAFFIC/labels
20240617185734.txt 20240617185957.txt 20240617190209.txt 20240617190210.txt
20240617185736.txt 20240617190000.txt 20240617190211.txt 20240617190212.txt
20240617185738.txt 20240617190003.txt 20240617190216.txt 20240617190217.txt
20240617185741.txt 20240617190005.txt 20240617190218.txt 20240617190219.txt
20240617185744.txt 20240617190007.txt 20240617190222.txt 20240617190223.txt
20240617185746.txt 20240617190009.txt 20240617190227.txt 20240617190228.txt
20240617185748.txt 20240617190011.txt 20240617190230.txt 20240617190231.txt
20240617185749.txt 20240617190014.txt 20240617190231.txt 20240617190232.txt
20240617185753.txt 20240617190018.txt 20240617190233.txt 20240617190234.txt
20240617185759.txt 20240617190021.txt 20240617190238.txt 20240617190239.txt
20240617185804.txt 20240617190024.txt 20240617190240.txt 20240617190241.txt
20240617185807.txt 20240617190028.txt 20240617190242.txt 20240617190243.txt
20240617185810.txt 20240617190033.txt 20240617190244.txt 20240617190245.txt
20240617185814.txt 20240617190036.txt 20240617190245.txt 20240617190246.txt
20240617185817.txt 20240617190042.txt 20240617190248.txt 20240617190249.txt
20240617185819.txt 20240617190044.txt 20240617190252.txt 20240617190253.txt
20240617185821.txt 20240617190050.txt 20240617190253.txt 20240617190254.txt
20240617185824.txt 20240617190055.txt 20240617190256.txt 20240617190257.txt
```

在 images 目录中存放的是所有的图片，在 labels 目录中存放的是标注文件，和图片是一一对应的，文件名相同，只是后缀名不同。

2. 我们提供一个拆分数数据集的脚本：

```
import os

import random

import shutil


# 设置随机数种子

random.seed(123)


# 定义文件夹路径

root_dir = 'TRAFFIC'

image_dir = os.path.join(root_dir, 'images')

label_dir = os.path.join(root_dir, 'labels')

output_dir = 'traffic_dataset'


# 定义训练集、验证集和测试集比例

train_ratio = 0.7

valid_ratio = 0.15

test_ratio = 0.15


# 获取所有图像文件和标签文件的文件名（不包括文件扩展名）

image_filenames = [os.path.splitext(f)[0] for f in os.listdir(image_dir)]

label_filenames = [os.path.splitext(f)[0] for f in os.listdir(label_dir)]


# 随机打乱文件名列表
```



```
random.shuffle(image_filenames)

# 计算训练集、验证集和测试集的数量

total_count = len(image_filenames)

train_count = int(total_count * train_ratio)

valid_count = int(total_count * valid_ratio)

test_count = total_count - train_count - valid_count


# 定义输出文件夹路径

train_image_dir = os.path.join(output_dir, 'train', 'images')

train_label_dir = os.path.join(output_dir, 'train', 'labels')

valid_image_dir = os.path.join(output_dir, 'valid', 'images')

valid_label_dir = os.path.join(output_dir, 'valid', 'labels')

test_image_dir = os.path.join(output_dir, 'test', 'images')

test_label_dir = os.path.join(output_dir, 'test', 'labels')


# 创建输出文件夹

os.makedirs(train_image_dir, exist_ok=True)

os.makedirs(train_label_dir, exist_ok=True)

os.makedirs(valid_image_dir, exist_ok=True)

os.makedirs(valid_label_dir, exist_ok=True)

os.makedirs(test_image_dir, exist_ok=True)

os.makedirs(test_label_dir, exist_ok=True)
```

```
# 将图像和标签文件划分到不同的数据集中

for i, filename in enumerate(image_filenames):

    if i < train_count:

        output_image_dir = train_image_dir

        output_label_dir = train_label_dir

    elif i < train_count + valid_count:

        output_image_dir = valid_image_dir
```

该文件 split_dataset.py 放在 TRAFFIC 目录的同级目录中。

3. 使用如下命令运行该程序，进行数据集拆分：

```
python3 split_dataset.py
```

执行成功后，在当前目录下生成了一个 traffic_dataset 目录：

```
(yolov5-7) lebox100@lebox100:~/data$ tree -L 2  traffic_dataset/

traffic_dataset/
├── test
│   ├── images
│   └── labels
├── train
│   ├── images
│   └── labels
└── valid
    ├── images
    └── labels
```

该目录下又分成了三个子目录 test、train、valid，分别对应了测试集、训练集、验证集。

4. 对 YOLOv5 进行数据集配置

接下来，我们对 YOLOv5 的数据集进行配置，为训练模型做准备。

我们在 data 目录中创建一个 voc_traffic.yaml 文件，该文件参考 coco.yaml 文件格式编辑内容如下：

```
train: ../data/traffic_dataset/train/images

val: ../data/traffic_dataset/valid/images

test: ../data/traffic_dataset/test/images


# number of classes

nc: 3

# class names

names: [ 'turnleft', 'turnright', 'turnaround']
```

5. 修改训练程序，使用自定义的训练数据配置

修改 train.py 文件中如下内容：

```
parser.add_argument('--data',          type=str,          default='data/voc_traffic.yaml',
                    help='data.yaml path')
```

6.4 任务四 YOLOv5 模型训练

1. 首先确保 Anaconda 的虚拟环境切换到 yolov5-7

在终端中输入如下指令：

```
lebox100:~$ conda activate yolov5-7

(yolov5-7) lebox100@lebox100:~$
```

2. 到 yolov5-7 目录下运行模型训练

```
cd yolov5-7

python3 train.py
```

可以看到，开始进行模型训练：

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
(yolov5-7) lebox100@lebox100:~/yolov5-7.0$ python3 train.py
train: weights=yolov5s.pt, cfgs=data/data/voc_traffic.yaml, hyp=data/hyps/hyp.scratch-low.yaml, epochs=100, batch_size=16, imgsz=640, rect=False, resume=False, nosave=False, noval=False, noautoanchor=False,
noplots=False, evolve=None, bucket=None, image_weights=False, device=None, multi_scale=False, single_cls=False, optimizer=SGD, sync_bn=False, workers=8, project=runs/train, name=exp, exist_ok=False,
quad=False, cos_lr=False, label_smoothing=0.0, patience=100, freeze=[0], save_period=1, seed=0, local_rank=-1, entity=None, upload_dataset=False, bbox_interval=1, artifact_alias=latest
github: skipping check (not a git repository), for updates see https://github.com/ultralytics/yolov5
YOLOv5 2022-11-22 Python-3.8.20 torch-2.4.1+cu121 CPU

hyperparameters: lr=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=0.05, cls=0.5, cls_pw=1.0, obj=1.0, obj_pw=1.0, iou_t=0.2, anchor_
t=4.0, fl_gamma=0.0, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0, perspective=0.0, flipud=0.0, fliplr=0.5, mosaic=1.0, mixup=0.0, copy_paste=0.0
clearml: run 'pip install clearml' to automatically track, visualize and remotely train YOLOv5 in ClearML
powers: run 'pip install comet ml' to automatically track and visualize YOLOv5 runs in Comet
TensorBoard: Start with 'tensorboard --logdir runs/train', view at http://localhost:6006/
train.py:123: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle
data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weig
hts_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly a
llowed by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open an
issue on GitHub for any issues related to this experimental feature.
ckpt = torch.load(weights, map_location='cpu') # load checkpoint to CPU to avoid CUDA memory leak
Overriding model.yaml nc=80 with nc=3

   from n  params module                        arguments
   --
0      -1 1      3520 models.common.Conv          [3, 32, 6, 2, 2]
1      -1 1     18560 models.common.Conv          [32, 64, 3, 2]
2      -1 1     18816 models.common.C3            [64, 64, 1]
3      -1 1     73984 models.common.Conv          [64, 128, 3, 2]
4      -1 2    115712 models.common.C3            [128, 128, 2]
5      -1 1    295424 models.common.Conv          [128, 256, 3, 2]
6      -1 3    625152 models.common.C3            [256, 256, 3]
7      -1 1    1180672 models.common.Conv          [256, 512, 3, 2]
8      -1 1    1182720 models.common.C3            [512, 512, 1]
9      -1 1    656896 models.common.SPPF          [512, 512, 5]
10     -1 1    131584 models.common.Conv          [512, 256, 1, 1]
11     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12     [-1, 6] 1      0 models.common.Concat          [1]
13     -1 1    361984 models.common.C3            [512, 256, 1, False]
14     -1 1    33024 models.common.Conv          [256, 128, 1, 1]
15     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16     [-1, 4] 1      0 models.common.Concat          [1]
17     -1 1    90880 models.common.C3            [256, 128, 1, False]
18     -1 1    147712 models.common.Conv          [128, 128, 3, 2]
19     [-1, 14] 1      0 models.common.Concat          [1]
20     -1 1    296448 models.common.C3            [256, 256, 1, False]
21     -1 1    590336 models.common.Conv          [256, 256, 3, 2]
22     [-1, 10] 1      0 models.common.Concat          [1]
23     -1 1    1182720 models.common.C3            [512, 512, 1, False]
24     [17, 20, 23] 1 21576 models.yolo.Detect      [3, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119], [116, 90, 156, 198, 373, 326]], [128, 256, 512]]

Model summary: 214 Layers, 702720 parameters, 702720 gradients, 16.0 GFLOPs

Transferred 343/349 items from yolov5s.pt
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
train: Scanning /home/lebox100/data/traffic_dataset/train/labels... 210 images, 0 backgrounds, 0 corrupt: 100%|██████████| 210/210 00:00
train: New cache created: /home/lebox100/data/traffic_dataset/train/labels.cache
val: Scanning /home/lebox100/data/traffic_dataset/valid/labels... 45 images, 0 backgrounds, 0 corrupt: 100%|██████████| 45/45 00:00
val: New cache created: /home/lebox100/data/traffic_dataset/valid/labels.cache
```

经过一段时间的训练之后，当训练达到预定的轮次，将得到训练后的模型。

3. 到训练结果目录进行结果查看：

```
(yolov5-7) lebox100@lebox100:~/yolov5-7.0/runs/train/exp49$ ls -la

总用量 3824

drwxrwxr-x  3 lebox100 lebox100  4096 7月  31 20:10 .

drwxrwxr-x 49 lebox100 lebox100  4096 7月  31 19:04 ..

-rw-rw-r--  1 lebox100 lebox100 94233 7月  31 20:10 confusion_matrix.png

-rw-rw-r--  1 lebox100 lebox100 669283 7月  31 20:10
```

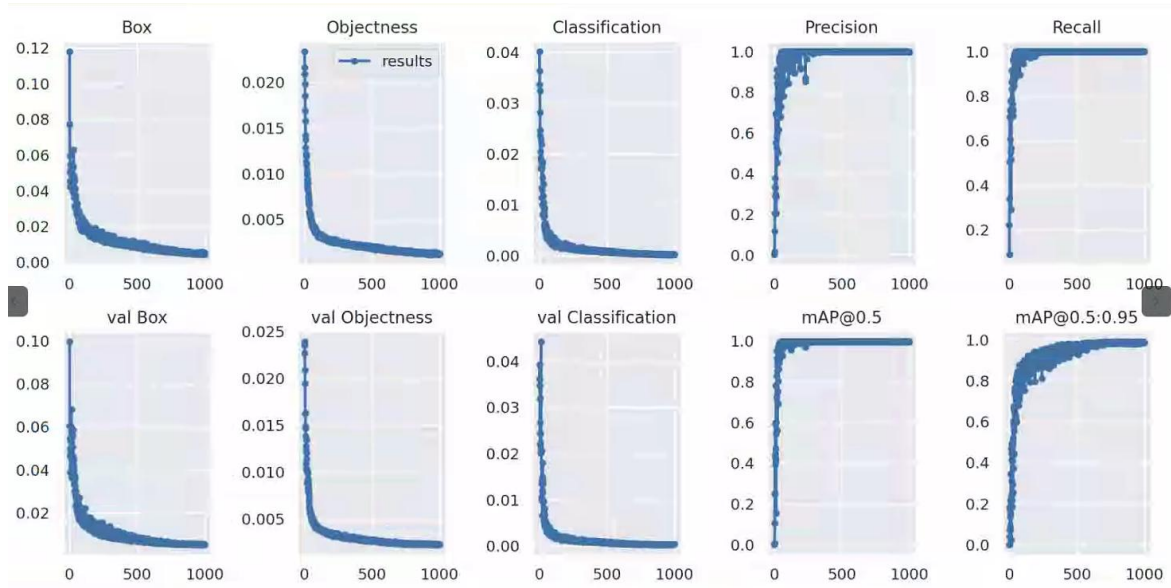
events.out.tfevents.1722423862.lebox100.8605.0

```
-rw-rw-r-- 1 lebox100 lebox100 102975 7 月 31 20:10 F1_curve.png
-rw-rw-r-- 1 lebox100 lebox100 356 7 月 31 19:04 hyp.yaml
-rw-rw-r-- 1 lebox100 lebox100 216321 7 月 31 19:04 labels_correlogram.jpg
-rw-rw-r-- 1 lebox100 lebox100 131985 7 月 31 19:04 labels.jpg
-rw-rw-r-- 1 lebox100 lebox100 665 7 月 31 19:04 opt.yaml
-rw-rw-r-- 1 lebox100 lebox100 82802 7 月 31 20:10 P_curve.png
-rw-rw-r-- 1 lebox100 lebox100 80056 7 月 31 20:10 PR_curve.png
-rw-rw-r-- 1 lebox100 lebox100 98698 7 月 31 20:10 R_curve.png
-rw-rw-r-- 1 lebox100 lebox100 194528 7 月 31 20:10 results.png
-rw-rw-r-- 1 lebox100 lebox100 151000 7 月 31 20:10 results.txt
-rw-rw-r-- 1 lebox100 lebox100 236658 7 月 31 20:10 test_batch0_labels.jpg
-rw-rw-r-- 1 lebox100 lebox100 238537 7 月 31 20:10 test_batch0_pred.jpg
-rw-rw-r-- 1 lebox100 lebox100 222594 7 月 31 20:10 test_batch1_labels.jpg
-rw-rw-r-- 1 lebox100 lebox100 224555 7 月 31 20:10 test_batch1_pred.jpg
-rw-rw-r-- 1 lebox100 lebox100 255978 7 月 31 20:10 test_batch2_labels.jpg
-rw-rw-r-- 1 lebox100 lebox100 257702 7 月 31 20:10 test_batch2_pred.jpg
-rw-rw-r-- 1 lebox100 lebox100 174477 7 月 31 19:04 train_batch0.jpg
-rw-rw-r-- 1 lebox100 lebox100 200523 7 月 31 19:04 train_batch1.jpg
```

```
-rw-rw-r-- 1 lebox100 lebox100 214766 7月 31 19:04 train_batch2.jpg
```

使用如下指令查看：

```
xdg-open results.png
```



可以看到训练结果比较理想，mAP 值都接近于 1.0。

使用如下指令，可以看到训练得到的模型 best.pt

```
(yolov5-7) lebox100@lebox100:~/yolov5-7.0/runs/train/exp49$ ls weights/  
best.pt  last.pt
```

4. 可以对模型效果进行测试：

修改 detect.py 文件如下内容

```
parser.add_argument('--weights',                    nargs='+',                    type=str,  
                    default='runs/train/exp49/weights/best.pt',  
                    help='model.pt path(s)')  
  
parser.add_argument('--source', type=str, default='../traffic_dataset/test/images',
```



```
help='source') # file/folder, 0 for webcam
```

执行如下指令进行推理测试：

```
python3 detect.py
```

可以看到，程序对每张图片进行推理

```
Namespace(agnostic_nms=False, augment=False, classes=None, conf_thres=0.25, device='0', exist_ok=False, img_size=640, iou_thres=0.5,
txt=False, source='../data/traffic_dataset/test/images/', update=False, view_img=False, weights='runs/train/exp49/weights/best.pt')
YOLOV5 2021-4-12 torch 1.12.1 CUDA:0 (NVIDIA GeForce RTX 3060, 12050.6875MB)

Fusing layers...
/home/lebox100/.conda/envs/pytorch/lib/python3.8/site-packages/torch/functional.py:478: UserWarning: torch.meshgrid: in an upcoming
version, this will be replaced with torch.meshgrid(...). Internally at /opt/conda/conda-bld/pytorch_1659484810403/work/aten/src/ATen/native/TensorShape.cpp:2894.)
  return _VF.meshgrid(tensors, **kwargs) # type: ignore[attr-defined]
Model Summary: 224 layers, 7059304 parameters, 0 gradients, 16.3 GFLOPS
image 1/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617185744.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnleft, Done. (0.029s)
image 2/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617185759.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnleft, Done. (0.004s)
image 3/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617185840.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnleft, Done. (0.004s)
image 4/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617190000.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnright, Done. (0.004s)
image 5/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617190003.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnright, Done. (0.004s)
image 6/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617190009.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnright, Done. (0.004s)
image 7/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617190050.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.004s)
image 42/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617192104.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.005s)
image 43/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617192115.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.004s)
image 44/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617192142.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.004s)
image 45/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617192150.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.004s)
image 46/46 /home/lebox100/yolov5-5.0/./data/traffic_dataset/test/images/20240617192156.jpg: torch.Size([1, 3, 480, 640])
torch.Size([1, 18900, 8])
480x640 1 turnaround, Done. (0.004s)
Results saved to runs/detect/exp72
Done. (0.568s)
```

打开任意一张推理结果图片来查看：

```
cd runs/detect/exp72
xdg-open 20240617190538.jpg
```



可以看到交通标识被正确识别，且置信度高达 0.97。

7

实验结果

1. 了解了目标检测神经网络 YOLOv5 及其特点。
2. 了解了 YOLOv5 模型训练软硬件环境需求及搭建过程。
3. 掌握了 Anaconda 的安装过程及创建虚拟环境的过程。
4. 掌握了 YOLOv5 的下载及安装依赖的过程。
5. 掌握了对 YOLOv5 标注数据集进行拆分的方法。
6. 掌握了如何配置 YOLOv5 训练参数。
7. 掌握了如何进行 YOLOv5 训练，并对模型效果进行评估。

8

实验资源释放

1. 关闭终端窗口。
2. 关闭宿主机。

9

实验小结

通过准备 YOLOv5 模型训练的软硬件环境，让开发者了解到进行 YOLOv5 模型训练所需要的硬件设备、软件环境、yolov5-7.0 版本的下载及依赖安装过程，并通过数据集拆分让开发者了解到拆分方法和不同数据集的用途以及配置方法，通过 YOLOv5 模型训练让开发者了解到如何进行模型训练以及对训练结果进行效果评估。

该实验为下一步在 ROS 中开发目标检测应用打下基础，ROS 只需要在边缘端设备上调用本节训练出的 YOLOv5 模型即可进行目标检测。